

Plano de Testes (Test Plan) — Calculadora Web 4(5) Funções

QA e Dev: Daniel Felipe Moraes Schaurich

01 - OBJETIVO DOS TESTES

Validar o funcionamento da lógica matemática (+, -, x, /) desenvolvida em JavaScript e HTML para navegador web. O intuito é garantir que o sistema está funcionando certo e sem erros antes de liberar para uso, atestando que o programa lide corretamente com entradas válidas e exiba alertas adequados para entradas inválidas.

Uma calculadora é uma ferramenta de exatidão. Se ela entregar um cálculo errado ou quebrar silenciosamente o resultado, o produto perde totalmente a credibilidade. Validar o funcionamento correto e os erros garante uma boa experiência de uso.

02 - METODOLOGIA DE TESTE

Será adotada uma abordagem baseada em casos de uso como teste Exploratório, Teste de Caixa Preta, Teste de Caixa Branca e Teste de Caixa Cinza, testando por etapas: primeiro as partes pequenas (Testes de Unidade), depois a integração com a tela, e, por fim, o sistema rodando por completo na visão do usuário final.

Inserir dados no HTML e verificar a saída isola os problemas. Se testarmos tudo de uma vez e der erro, será difícil saber se o problema está na interface ou na matemática do JavaScript.

03 - ESCOPO E TESTES

O escopo foca nas funções principais e nas interações já desenvolvidas na interface.

- Incluídos: Operações básicas da calculadora, porcentagem, comportamento do sistema com campos em branco ou com caracteres não numéricos.
- Excluídos: Testes de responsividade em telas de celular (foco inicial é apenas na lógica do desktop), funções matemáticas avançadas (não implementadas) e testes de segurança/performance de servidor (aplicação roda apenas localmente no navegador).

O escopo fechado evita desperdício de tempo em testes de funcionalidades que ainda não foram feitas ou que não fazem parte do objetivo principal da versão deste sistema.

- Justificativa do Fluxo: O criador (agindo como QA) mapeia os cenários e registra os bugs na interface. Depois (agindo como Dev), vai ao código consertar. Retorna como QA para garantir que o conserto funcionou, e por fim, avalia se atende à necessidade final. Essa disciplina evita vícios de desenvolvimento.

08 - RISCOS E CONTINGÊNCIAS (Gestão de Riscos do Código)

- Risco 1: Limite Técnico do JavaScript (O bug do `1e+73`).
 - *Consequência*: Como o JS utiliza números de ponto flutuante de precisão dupla, cálculos excessivamente grandes perdem a precisão visual, frustrando o usuário.
 - *Contingência (Ação Corretiva)*: Implementar no HTML um limite máximo de caracteres nos campos de input (ex: `maxlength="15"`).
- Risco 2: Ausência de bloqueio para Divisão por Zero.
 - *Consequência*: O JavaScript não dá erro de tela ao dividir por zero, ele apenas retorna o texto `Infinity`. Isso confunde e demonstra falta de polimento.
 - *Contingência (Ação Corretiva)*: Mapear isso no código da função `dividir()`, criando um alerta caso `n2 === 0`.
- Justificativa de Foco: Se o projeto atrasar, a equipe deve focar primeiro em corrigir a Divisão por Zero (que é um erro lógico e matemático), deixando ajustes de layout (como tamanho dos botões) para um segundo momento.

09 - FERRAMENTAS DE TESTE

- Navegador Web Microsoft Edge + Console (F12): Utilizados para os testes manuais e para verificar se algum erro interno de script é gerado silenciosamente ao clicar nos botões.
- Cypress / Playwright: Programas utilizados para automatizar os testes de tela (E2E), clicando nos botões sozinhos. (*Não executado)

10 - MÉTRICAS DE QUALIDADE

Para atestar o Go/No-Go (se a calculadora está pronta para uso real), acompanharemos:

- Taxa de Sucesso Funcional: Exigência de 100% de acerto nas contas com números convencionais (até 10 dígitos).
- Densidade de Bugs (Status Atual): Monitorar quantos testes passaram e quantos falharam. Atualmente, temos 1 bug de formatação mapeado (Notação Científica extrema) e 1 vulnerabilidade lógica aguardando correção (ausência de bloqueio na divisão por zero).

- Justificativa: As métricas transformam "achismos" em dados concretos. O sistema só avança para o usuário quando o número de bugs críticos e bloqueadores chegar a zero.

11 - RELATÓRIO DE FALHAS

Alguns testes foram realizados, inclusive “forçando” o erro no código para testar o funcionamento das operações e condições, porém não houve tempo hábil para resolver a questão do número inteiro e também a alteração do nome do arquivo, via bash, para a versão 1.2 (v1.2).